



Hands-On With the Neo4j Graph Platform Session 6



Class Setup

- Aura -unpause your instance (**Highly recommended**)
 - Neo4j Desktop -Start your DB
 - Neo4j Sandbox -make sure instance is started
- 

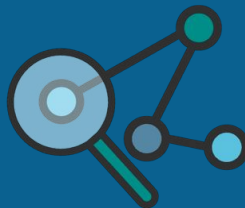
Class Agenda

- Labeled Property Graph (LPG) Components 🙌
- What is a graph database? 🙌
- What is Neo4j? 🙌
- Common Graph Use Cases 🙌
- Graph modeling -Part 1 🙌

➤ Graph modeling - Part 2 🙌

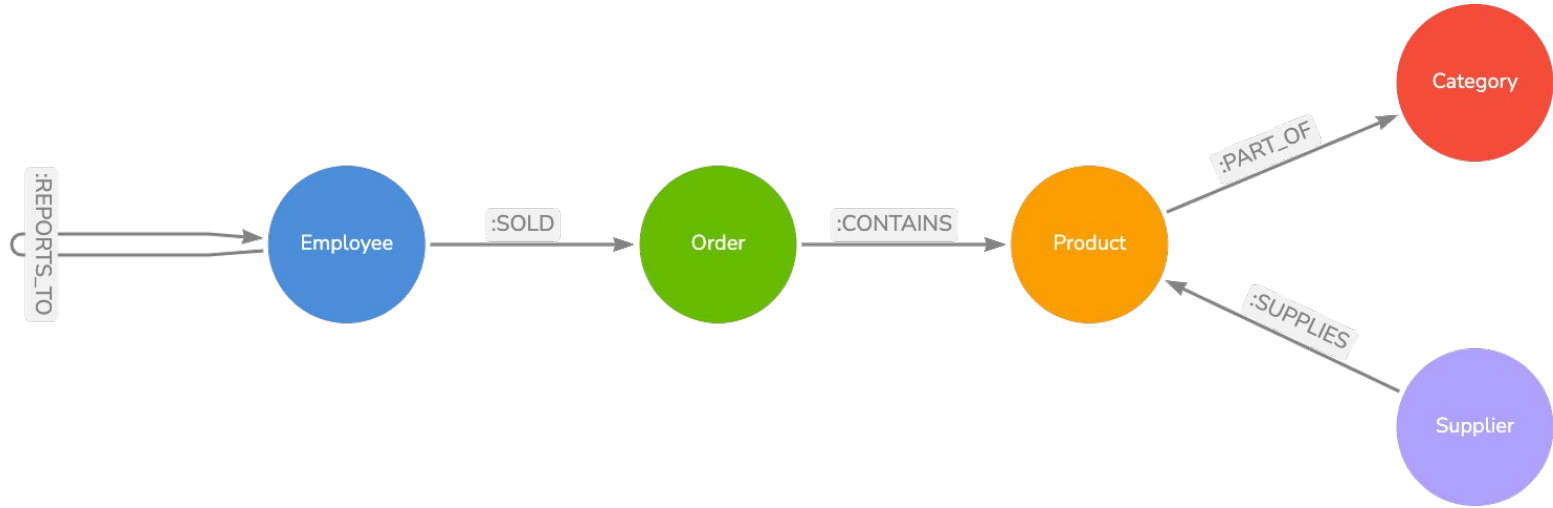
- Importing CSV into Neo4j
- Constraints and Indexes
- Intro to Graph Data Science
- Graph Data Science workflows

🙌 Hands-On



Modeling Your Data As A Graph Exercise (continued)

Data Model - Relationships



CALL db.schema.visualization

Relationships Normalize Data

Label :Person
Property A Person's primary interest
Relationship :HAS

Denormalized:

```
CREATE (:Person {name:'David', interests: ['Guitar', 'Bosco']})  
CREATE (:Person {name:'Jessica', interests: ['Guitar']})
```

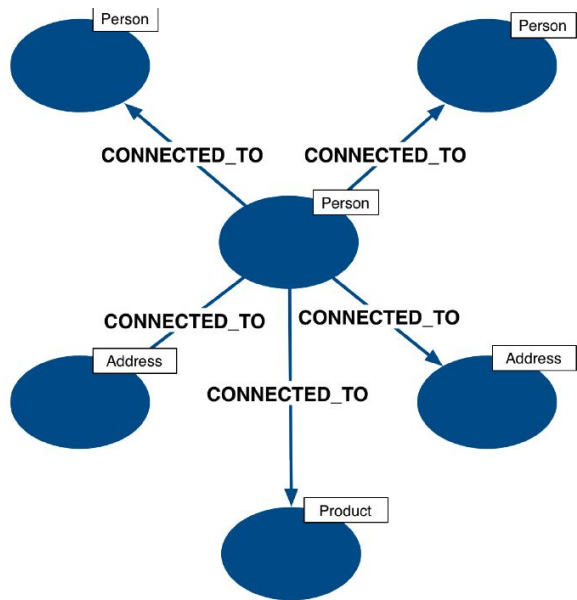
Normalized:

```
CREATE (ig:Interest {name: 'Guitar'}), (ib:Interest {name: 'Bosco'})  
CREATE (p1:Person {name:'David'})  
CREATE (p2:Person {name:'Jessica'})  
CREATE (p1)-[:HAS]->(ig), (p1)-[:HAS]->(ib)  
CREATE (p2)-[:HAS]->(ig)
```

Relationship Semantics

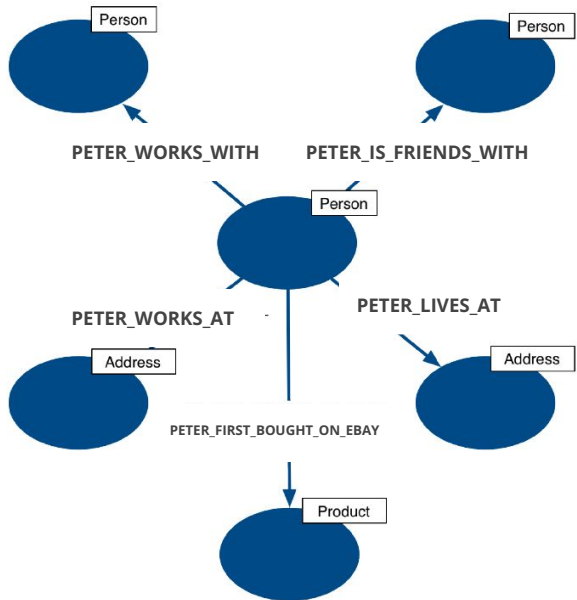
Composition / "HAS A"	(:Person)-[:HAS_INTEREST]->(ig)
Inheritance / "IS A"	(p)-[:IS_A]->(:Employee)
Functional / "HAS <VERB>"	(:Person)-[:HAS_HOME_ADDR]->()
Transitive / "IS <VERB>"	(:Person)-[:IS_RELATED_TO]->()
Reflexive / "<VERB>"	(:Person:)-[:KNOWS]->(:Person)
Symmetric / "IS <VERB>"	(:Person)-[:MARRIED]-(:Person)
Vanilla / "XXX"	(:Person)-[:UNKNOWN]->(:Item)

...but not too specific



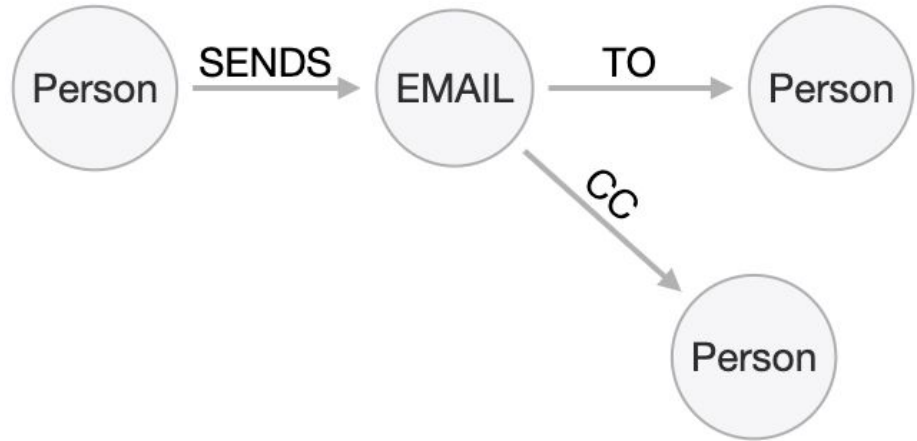
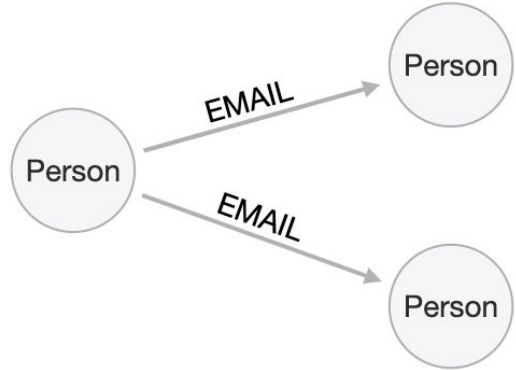
Not specific enough

Too specific



Naming Relationships

- Stakeholders must agree upon name
- Avoid names that could be construed as nouns (for example **email**)
- Try to avoid generic relationships, unless necessary (such as 'UNKNOWN')



Deciding on Relationship Properties

1. Many Relationships will not have properties

2. Common Relationship properties

Path traversal data or metadata

- weight, cost, distance used in rollup and compare
- GDS pathfinding algorithms

3. Administrative metadata:

- version

Data Modeling Tips

- There is no right or wrong way to model your data.
- Some ways may be better suited to your needs and more performant on the aspects you prioritize.
- Finding the best data model for your needs, it often helps to approach with a few techniques and make data model decisions from that analysis.



Creating Nodes and Relationships

Cypher for Create & Delete

CREATE - Used to create new nodes

- Uses MAP syntax
- For example

```
MATCH (ah:AccountHolder)
WHERE ah.accountHolderID=1234
CREATE (prof:AccountProfile {
  accountHolderID: ah.accountHolderID
  , profileDate: datetime()
  , lastName: ah.lastName
  , firstName: ah.firstName
  , accountName: ah.accountName
})
```

```
MERGE (ah)-[:HAS_PROFILE]->(prof)
```

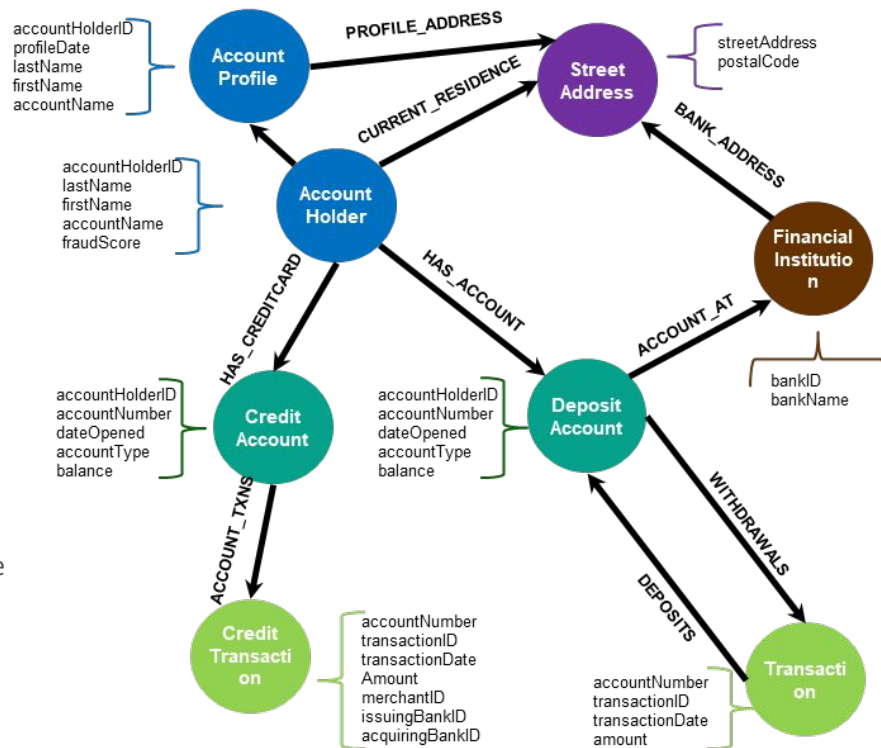
- This is one statement and one transaction

(DETACH) DELETE

- Removes a node (if no relationships) or a relationship
- If DETACH DELETE, first removes relationships, then drops node

```
MATCH (ah:AccountHolder)
WHERE ah.accountHolderID=5678
DETACH DELETE ah
```

```
:auto MATCH (ah:AccountHolder)
CALL {
  WITH ah
  DETACH DELETE ah
} IN TRANSACTIONS OF 1000 ROWS
```



CREATE vs. MERGE

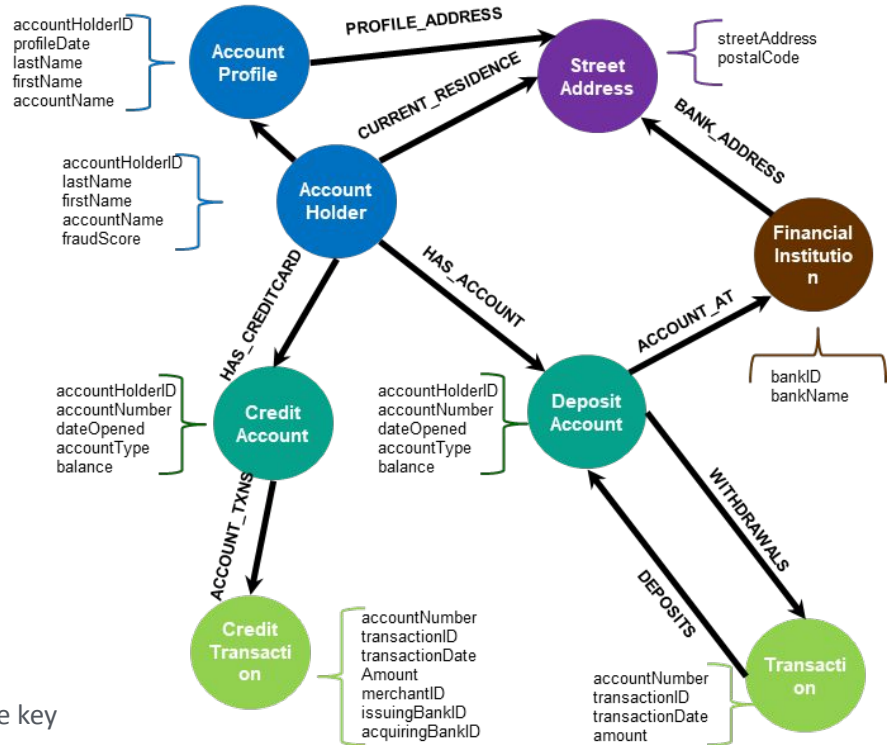
An alternative to CREATE is MERGE

- Functions like UPSERT in ORACLE or SQL MERGE
 - *Creates if it doesn't exist, updates if it does*
- Uses JSON syntax for keys to match on
- For example

```
MATCH (ah:AccountHolder)
WHERE ah.accountHolderID=1234
MERGE (prof:AccountProfile {
  accountHolderID: ah.accountHolderID
, profileDate: datetime()})
// ON MATCH SET ...
ON CREATE SET prof.lastName=ah.lastName
, prof.firstName=ah.firstName
, prof.accountName=ah.accountName
MERGE (ah)-[HAS_PROFILE]->(prof)
```
- This is still one statement and one transaction

CREATE vs. MERGE

- Since MERGE is doing a search, it really needs an index (or node key constraint) or else node scan (ala table scan)



Updating properties

SET clause

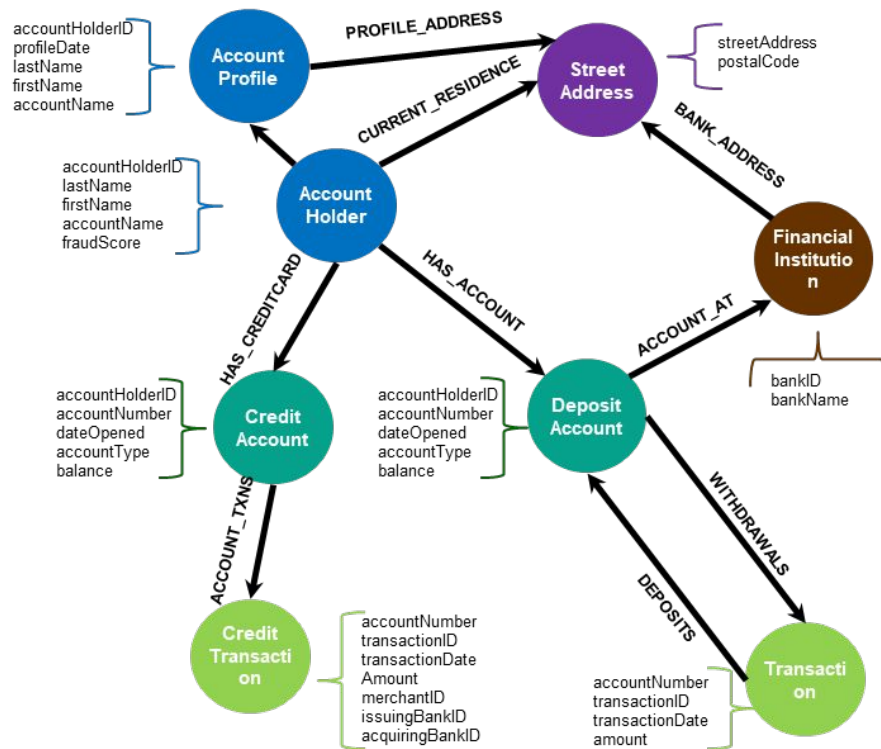
- Change someone's last name
`MATCH (ah:AccountHolder)`
`WHERE accountHolderID=1234`
`SET ah.lastName="SomethingNew"`
- This also works on relationships (and other things)

SET is not restricted to just single node

```
CREATE (seq:Sequence {accountHolderID: 0});  
// ...time goes on...  
MATCH (seq:Sequence)  
MERGE (ah:AccountHolder  
  {accountHolderID: seq.accountHolderID+1}  
ON CREATE SET ah.lastName="Smith"  
  , ah.firstName="John"  
  , ah.accountName="Smith, John"  
  , ah.fraudScore=0.5
```

MERGE + SET = update (or create) regardless

```
MATCH (seq:Sequence)  
MERGE (ah:AccountHolder  
  {accountHolderID: seq.accountHolderID+1}  
SET ah.lastName="Smith"  
  , ah.firstName="John"  
  , ah.accountName="Smith, John"  
  , ah.fraudScore=0.5
```



Updating relationships

SET clause

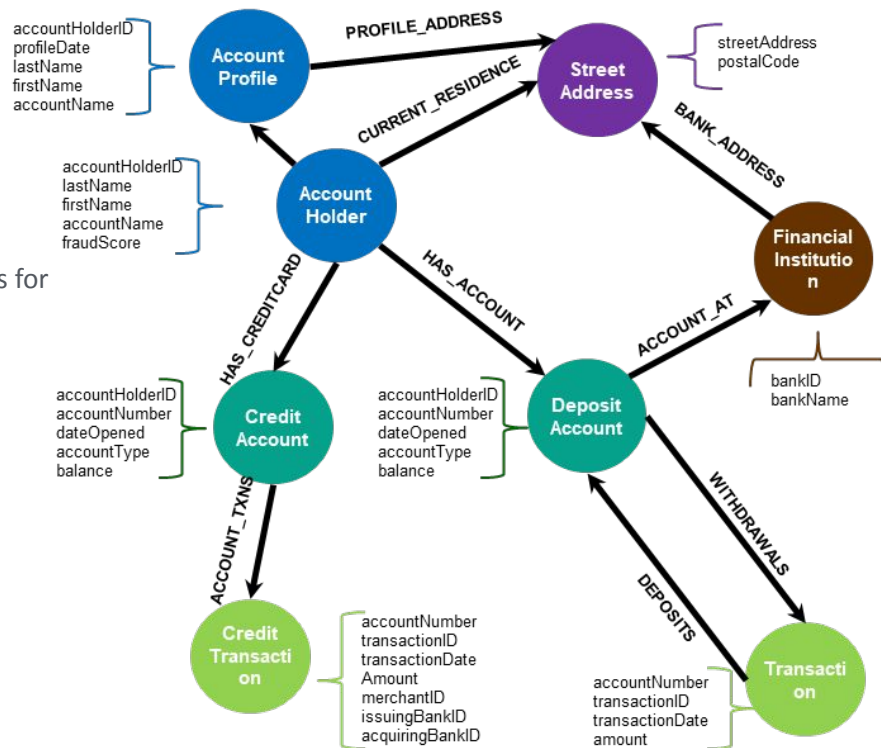
- To change or add a property on a relationship

```
MATCH (:AccountHolder)-[cr:CURRENT_RESIDENCE]->(:StreetAddress)
WHERE accountHolderID=1234
SET cr.fromDate=date("2015-01-15")
```

But...may not be best choice

- Customer moves but we want to keep track of their old address for profile reasons (and to detect account takeover attempts)
- Result may be a multi-clause statement

```
MATCH (ah:AccountHolder)
-[cr:CURRENT_RESIDENCE]->(addr:StreetAddress)
MERGE (prof:AccountProfile {accountHolderID: ah.accountHolderID
, profileDate=date()})
ON CREATE SET prof.lastName=...
MERGE (prof)-[:PROFILE_ADDRESS]->(addr)
DELETE cr
CREATE (newAddr:StreetAddress {...})
MERGE (ah)-[:CURRENT_RESIDENCE]->(newAddr)
```



Primary load command - LOAD CSV

Basic syntax is LOAD + CREATE

LOAD CSV is just a basic clause to stream data

- Which means just about any clause can follow
- E.g. nothing special about create in this instance – it is just the next clause in line
- Can load from public URL (non-authenticated)
 - If authentication required, APOC utilities, DW Connector, HOP, Talend or other tool can be used

Supports 2 special functions

- Linenumber()
 - *By itself, can be used to create row id's/node key values*
 - *Used with where clause, allows you to skip portions of file*
- Filename()
 - *Allows you to keep track of what files you've loaded via an automated (python) script*
 - *Create a node "LoadTracker" and as final MERGE in statement, create a new entry and datetime() for time (you can use WITH and datetime() at beginning to get a start time).*

```
// :auto only necessary from neo4j browser
[:auto] LOAD CSV [USING HEADERS] FROM <url> AS
<variable>
CREATE (:label {
    <propname>:    <variable>.<column>
    [, <propname>:    <variable>.<column>
    [, <propname>: ...]})
```

```
[:auto] LOAD CSV [USING HEADERS] FROM <url> AS <variable>
[WITH <variable>]
[WHERE <predicate cond> [LIMIT ###]]
CALL {
WITH <variable>
[WITH [WHERE] [LIMIT ###]]
[MATCH ...]
[CREATE ...]
[MERGE ...]
} IN TRANSACTIONS [OF ##### ROWS]
[RETURN ...]
```

LOAD CSV Tip: Load Nodes then load relationships

```
// Step 1 - Load the data - if we know this is "new" data, we can just call CREATE vs. MERGE
// Added 1500000 labels, created 1500000 nodes, set 7500000 properties, completed after 59321 ms.
USING PERIODIC COMMIT 1000
LOAD CSV WITH HEADERS FROM 'file:///atm_txns.csv' AS input
CALL {
    WITH input
    CREATE (:Cash:Transactions {
        transaction_id: input.transaction_id,
        transaction_date: input.transaction_date,
        amount: toFloat(input.amount),
        accountNumber: input.account_number,
        type: "DEPOSIT"
    }) IN TRANSACTIONS OF 10000 ROWS
;
// Step 2 - Do the relationships by rescanning the file
LOAD CSV WITH HEADERS FROM 'file:///atm_txns.csv' AS input
CALL {
    WITH input
    MATCH (myDate:CalendarDate {date: date(input.transaction_date)})
    MATCH (myTxn:Cash:Transactions {transaction_id: input.transaction_id})
    MATCH (myAcct:DepositAccount {accountNumber: input.account_number})
    MERGE (myTxn)-[:TXN_DATE]->(myDate)
    MERGE (myAcct)-[:DEPOSITS]->(txn)
} IN TRANSACTIONS OF 1000 ROWS
;
```

Refactoring / Evolving a Graph Data Model

Why refactor?

Data models can be optimized for one of four things:

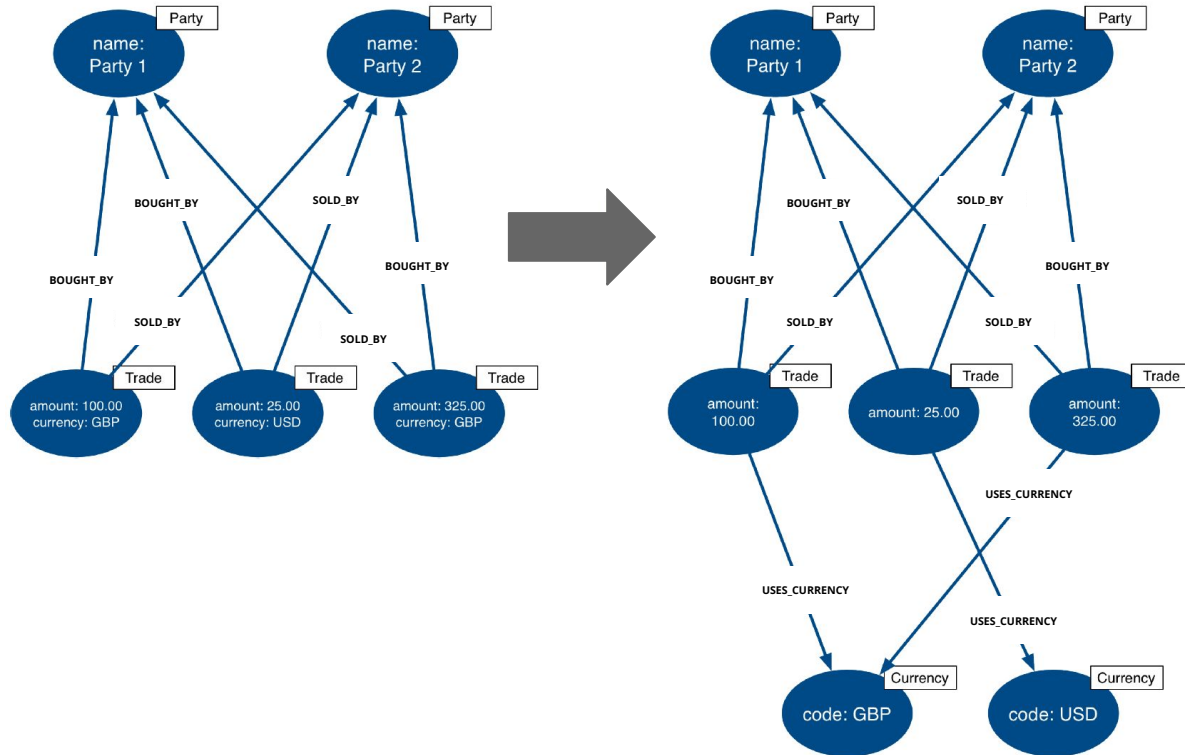
- Query performance
- Model simplicity & intuitiveness
- Query simplicity (i.e., simpler Cypher strings)
- Easy data updates

Improving behavior in one of these areas frequently involves sacrifices in others

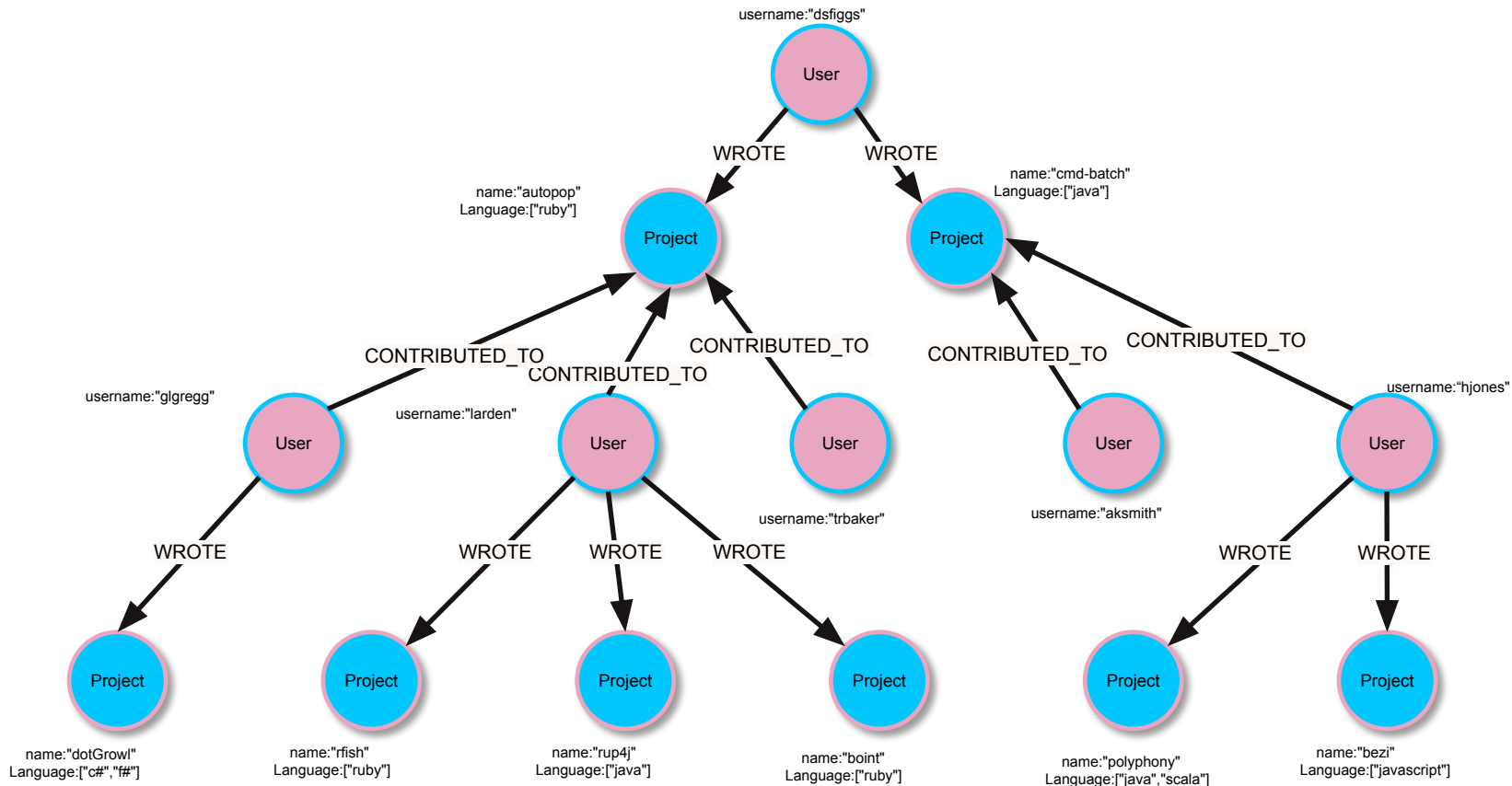
Another reason to refactor:

- Accommodating new application questions in the same model

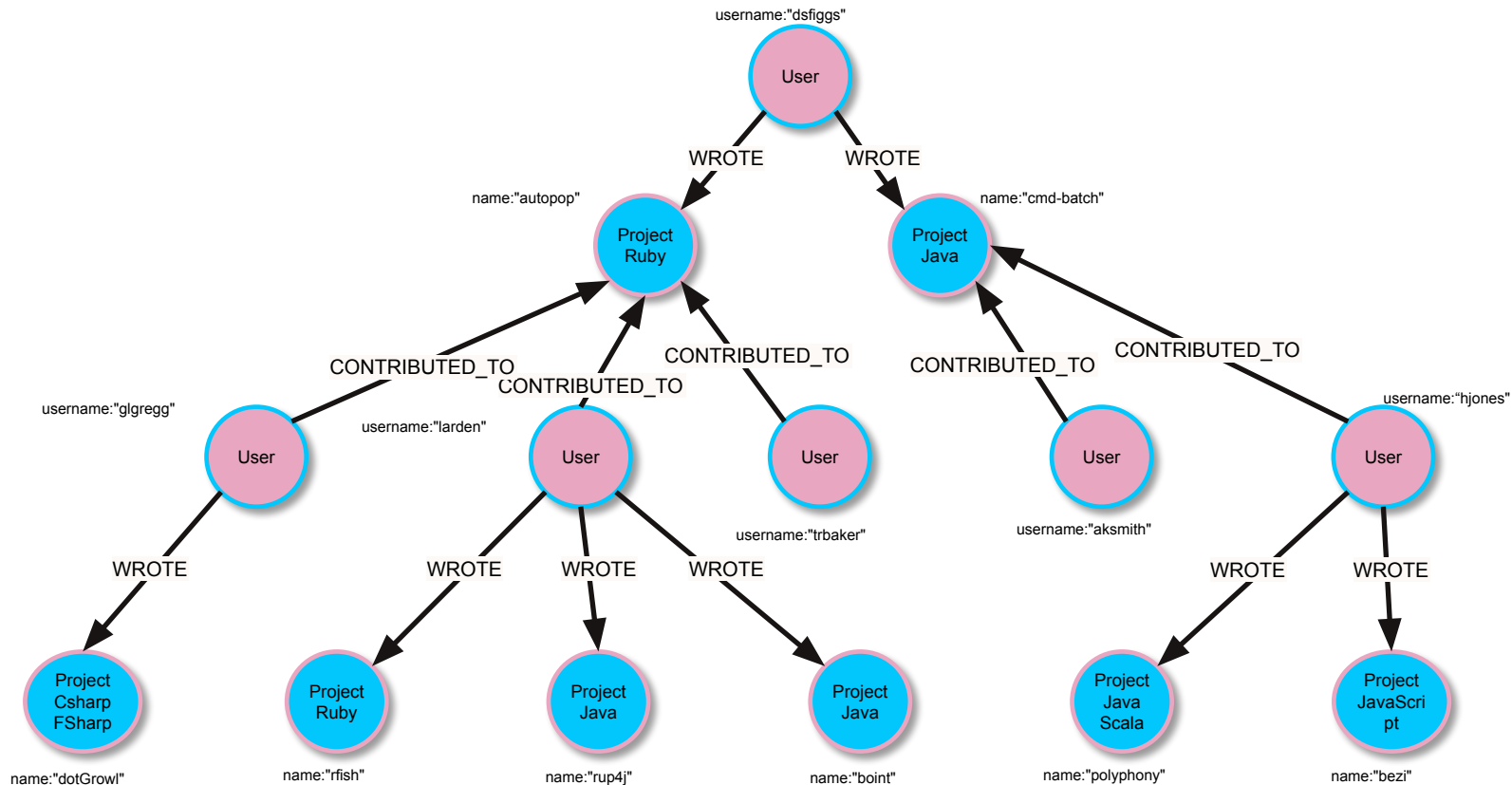
Example: Extract node from a property



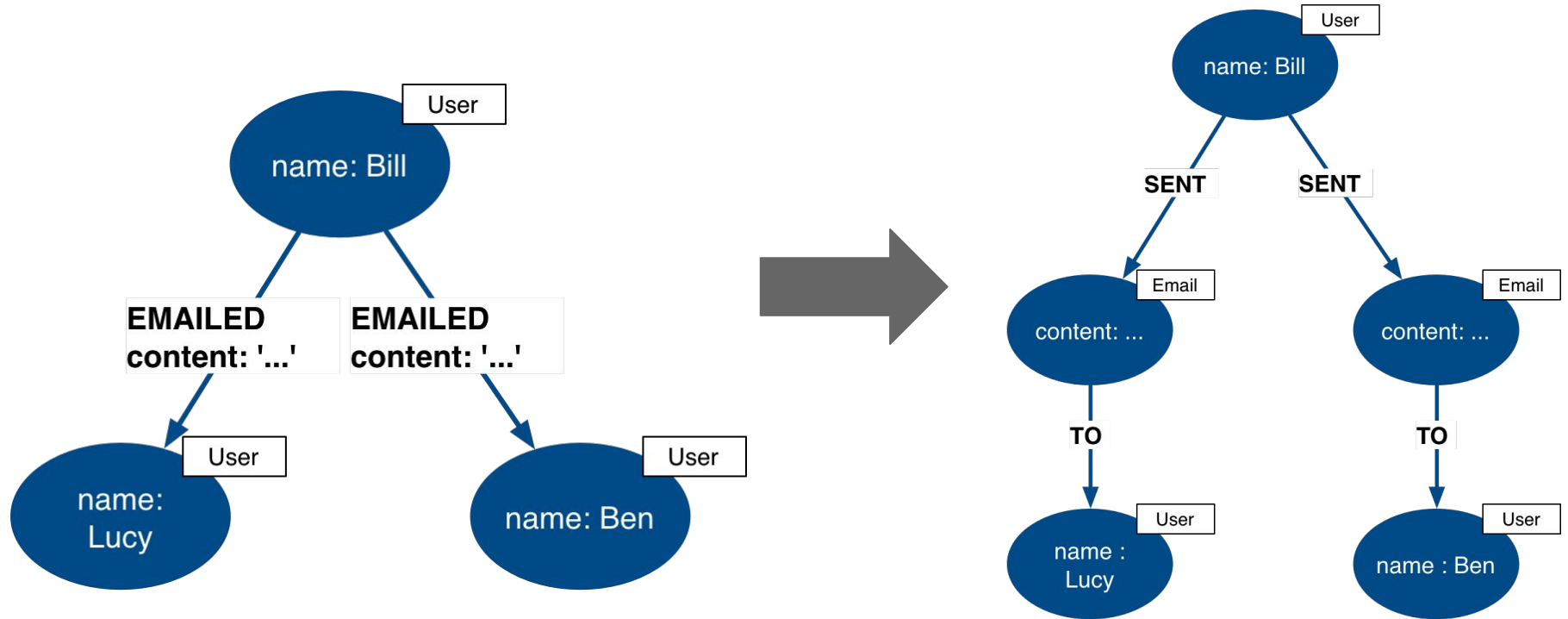
Example: Convert property to label (slide 1)



Example: Convert property to label (slide 2)



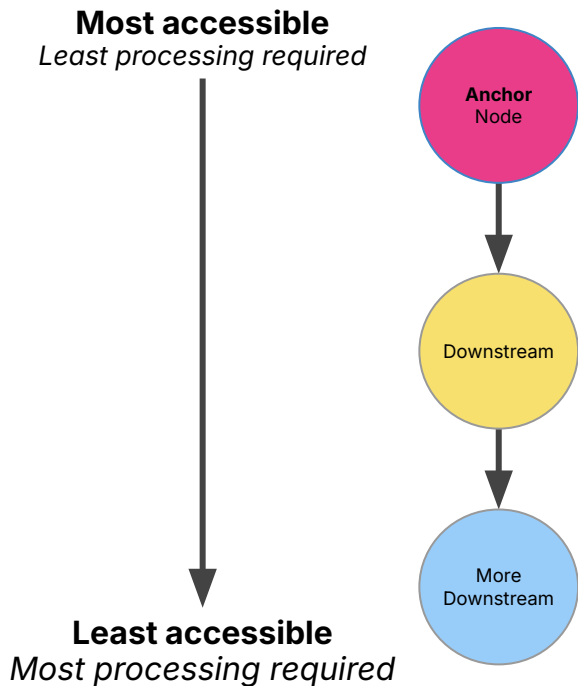
Example: Extract intermediate nodes



Consider: Hierarchy of Accessibility

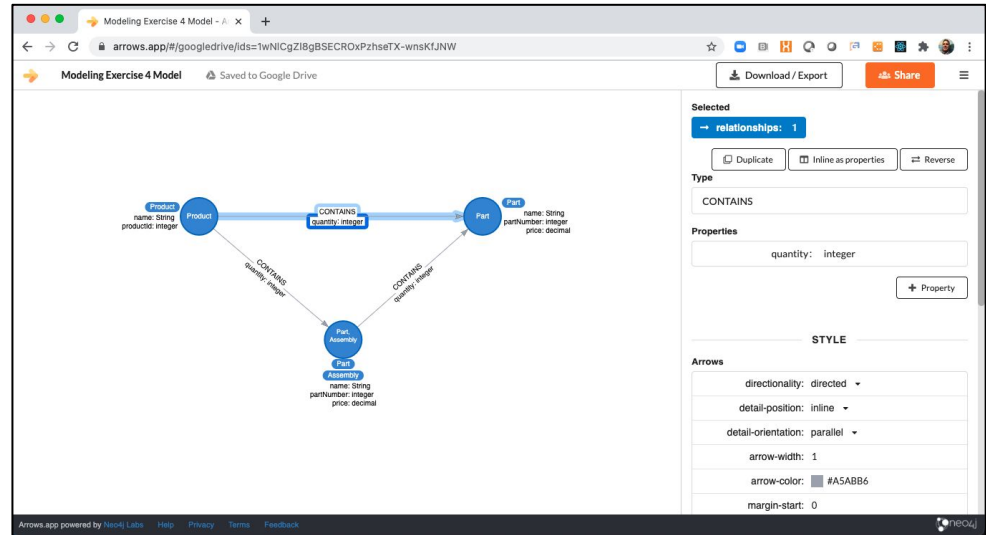
For each data object, how much work must Neo4j do to evaluate if this is a “good” path or a “bad” one?

1. **Anchor Node Label**
Anchor Node Properties (indexed)
2. **Anchor Relationship Type**
Anchor Relationship Properties (indexed)
3. Downstream **Node Labels**
Downstream **Relationship Type**
4. **Anchor Node Properties** (non-indexed)
Anchor Relationship Properties (non-indexed)
5. Downstream **Node Properties**
Downstream **Relationship Properties**



Arrows Tool @ arrows.app

- Quick, intuitive drawing with a mouse
- No cost, totally free
- Neo4j Property Graph Model
 - Draw: nodes, relationships, properties, labels
- Fine-grained Styling
 - Control: sizes, layouts, colors
- Export as image or Cypher
 - Use images in documents or presentations
 - Run Cypher to create graphs in Neo4j



[Arrows YouTube Video](#)

Data Importer: Getting started with modeling

Frictionless Experience



Easiest way to model your first graph and load it using flat files

- **Intuitive** data modeling UI
- **Map** flat files to graphs
- Built to import up to **millions of records**
- **Import directly** Neo4j Aura or self-managed

<https://medium.com/neo4j/importing-your-data-into-neo4j-just-got-even-easier-3712e531ca70>

<https://neo4j.com/docs/data-importer/current/overview/>



Hands On Time!

Run the Guidebook

Run:

:guide hands-on

Select:

Modeling and Importing Northwind data **(Finish)**

:guide hands-on/modeling-northwind.html

Cypher Manual

<https://neo4j.com/docs/cypher-manual/current/introduction/>

Cypher Cheat Sheet

<https://neo4j.com/docs/cypher-cheat-sheet/5/auradb-enterprise/>



Backups